Introducción a la Inteligencia Artificial y a los Sistemas de Aprendizaje Automático

Caso práctico



Max es una estudiante que está muy interesada en el campo de la inteligencia artificial. Parte de su aprendizaje se basa en la investigación de modelos y técnicas, pero ella lleva un tiempo preguntándose cómo y por qué surgió todo el ecosistema actual de la inteligencia artificial basada en el aprendizaje automático.

Recientemente, se ha lanzado una nueva película, y Max se conectó con sus amigos para comentarla. Se trata de una película en la que una inteligencia artificial tomaba conciencia de sí misma y trataba de controlar a los humanos que estaban en una nave espacial.

"¡Qué miedo! ¿No?" Exclamaba uno de ellos "Max, ¿realmente esto es posible? Tú deberías saberlo".

Max sonrió y contestó "Bueno, la verdad es que eso solo es posible con una inteligencia artificial de las que llamamos Fuerte o General, y por el momento, nadie ha conseguido nada a ese nivel.".

"Tengo un profesor que dice que eso de la inteligencia artificial solo es un nombre nuevo que le han dado a la Estadística para que parezca más interesante". Dice otro de los amigos de Max.

"Lo cierto es que el planteamiento actual de la inteligencia artificial se basa mucho en la Estadística y Probabilidad, pero ha evolucionado al fusionarlas con otras áreas como la Computación o técnicas nuevas como el Deep Learning."

(https://educacionadistancia.juntadeandalucia.es/formacionprofesional/pluginfile.php/139064/mod_scorm/content/0/MAX.jpg)

@casfatesvano (https://twitter.com/casfatesvano) (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

En esta unidad veremos:

- Introducción a la inteligencia artificial y las diferencias entre IA fuerte y débil.
- Caracterización de los sistemas de aprendizaje automático (aprendizaje supervisado y no supervisado).
- Herramientas para el aprendizaje supervisado (python y colab).
- Preprocesamiento y tratamiento de los datos.

1.- Inteligencia Artificial Fuerte y Débil

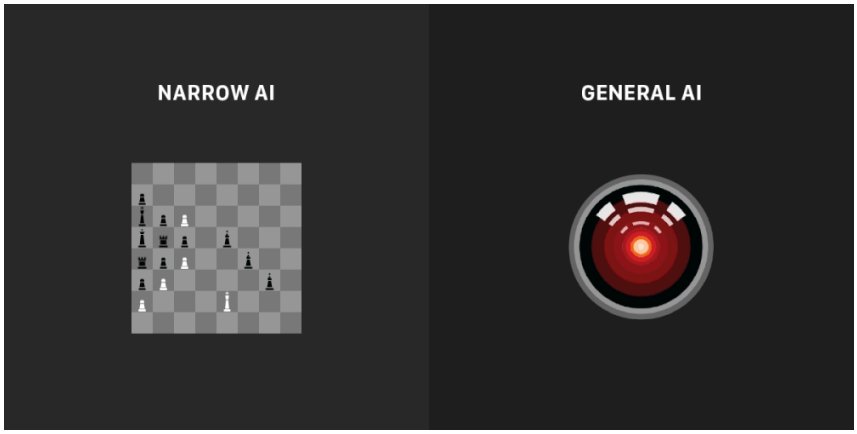
Caso práctico



@casfatesvano (https://twitter.com/casfatesvano) (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Max se ha quedado pensativa tras la conversación del otro día con sus amigos, cuando comentaban la película de una inteligencia artificial que trataba de dominar a los humanos. Se da cuenta de que, en realidad, no sabe mucho sobre los verdaderos límites de esta tecnología, y se dispone a investigar un poco más sobre ello.
Es cierto que la ciencia ficción nos muestra muchos casos de seres con inteligencia artificial que, incluso, se hacen pasar por humanos. La idea de un asistente personal que nos acompañe en todas nuestras tareas y que nos haga la vida más fácil parece realmente interesante. Max decide investigar si hay algún desarrollo en esa línea.

Aunque, en un principio, el objetivo del desarrollo de la inteligencia artificial era construir una máquina capaz de seguir las leyes de la lógica del pensamiento a la perfección y sin equivocarse, pronto se vio que esto era algo casi utópico. Tratar de programar todas las reglas necesarias, con sus propias correlaciones y generalizaciones, era una misión inabarcable y que siempre acababa en alguna incoherencia o fallo.
Los sistemas expertos, marcaron la clara separación entre dos tipos de inteligencia artificial. La inteligencia artificial fuerte o general, similar a la mente de los seres humanos, y la inteligencia artificial débil o estrecha, capaz de realizar con gran precisión una tarea concreta, dentro de un campo de especialización.
Toda la evolución que ha habido en el ámbito del aprendizaje automático profundo se ha desarrollado en el plano de la inteligencia artificial débil. Aun así, cada cierto tiempo, se van produciendo hitos y avances que vuelven a rescatar la idea de contar con una inteligencia artificial fuerte como la que aparece en las películas y novelas de ciencia ficción, capaz de relacionarse y servir a los humanos en modo multitarea.



Gavin Jensen (http://www.gavinjensen.com/blog/2018/ai-narrow-vs-general) (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Aunque la mayoría de alusiones a las diferencias entre ambas se refieren casi siempre a la capacidad de imitación de la inteligencia humana, esto no es del todo correcto. El campo de la inteligencia artificial actualmente no se centra en emular las capacidades o el comportamiento humano, a no ser que nos refiramos a robots de tipo humanoide o videojuegos, y ni siquiera sería del todo exacta esta concepción en esos casos. La clave de la inteligencia artificial actual es la evolución y avance de la automatización.
La mayoría de desarrollos actuales considerados como inteligencia artificial estrecha o débil superan con creces las capacidades humanas en ese ámbito de aplicación. La capacidad de abstracción y la creatividad de algunos modelos también empiezan a mostrar un alcance sorprendente.
En definitiva, es recomendable no basar la distinción entre ambas en la comparación con la inteligencia humana y centrarse más en la capacidad de realizar tareas en un único ámbito o contexto o ser capaz de abordar tareas muy diferentes en cuanto a entorno de aplicación y resultados.
A algunos expertos les gusta la idea de que la inteligencia artificial débil representa el punto en el que estamos ahora y que la inteligencia artificial fuerte o general es el siguiente paso evolutivo o el punto al que nos dirigimos.

Autoevaluación

La inteligencia artificial débil todavía no se ha conseguido llevar a la realidad, solo existe en la ciencia ficción.

○ Verdadero ○ Falso

Falso

En realidad es la inteligencia artificial fuerte la que todavía no se ha alcanzado.

1.1.- Inteligencia Artificial Débil

El concepto de inteligencia artificial débil se refiere a agentes y modelos inteligentes que están especializados en un tipo de tarea, dentro de un campo de especialización concreto. Por ejemplo, un ordenador que juega al ajedrez, o un modelo de reconocimiento de imagen.
Diseñar una inteligencia artificial de este tipo no es igual en el modelo clásico que en el paradigma actual de aprendizaje automático. Los sistemas expertos, que se programaban a base de reglas "if-else" del modelo clásico, eran más rígidos y no era nada sencillo actualizarlos o hacerlos evolucionar. Además, dependían mucho del equipo que lo diseñaba y construía, con el alto riesgo de introducir sesgos (https://www.ibm.com/es-es/topics/ai-bias)que eso conlleva.

Ventajas de la IA (Inteligencia Artificial) Débil:

- Actualmente, es relativamente sencilla de conseguir y desarrollar.
- Se controla más fácilmente y no presenta comportamientos demasiado inesperados.
- Permite arquitecturas modulares, con las que combinar distintos talentos o capacidades según las necesidades del entorno en el que se despliega.

Inconvenientes de la IA Débil:

- Es un sistema "ciego", que no cuenta con todo el contexto de la situación, por lo que puede malinterpretar entradas inusuales.
- Es muy limitada; solo sirve para aplicaciones muy concretas.
- Suelen ser sistemas que no ofrecen una experiencia de usuario tan cercana o personalizada como la de un ser humano.

En resumen, una IA Débil está concebida para ser aplicada en un ámbito específico de problemas, y no tanto de tener un comportamiento similar a la inteligencia humana. Desde este punto de vista, es impensable concebir que uno de estos sistemas constituyan una mente, un estado mental o conciencia propia.

Ejemplos de inteligencia artificial débil:

Alexa, Siri y Google Assistant

Estos modelos de asistentes virtuales, que están presentes en móviles y dispositivos para el hogar, son lo que se conoce como asistentes conversacionales. Son un ejemplo de que una inteligencia artificial estrecha puede simular un comportamiento humano, pero no ser capaz de llegar a dar el mismo resultado o ejecutar la misma tarea que la inteligencia humana proporcionaría.

IBM Watson

Este sistema cognitivo es capaz de dotar de la información precisa y necesaria en cada momento bajo demanda, y en un contexto de conversación en lenguaje natural o coloquial. Puede analizar gran cantidad de datos y extrapolar o inferir conclusiones a partir de ellos.

AlphaGo

Este sistema desarrollado por DeepMind ha sido capaz de batir en el juego Go al campeón mundial, como hizo Deep Blue (IBM) en su día con el campeón Kasparov. Otras versiones del mismo sistema han sido AlphaStar, capaz de jugar a un conocido videojuego, el Starcraft, y AlphaZero, una generalización de AlphaGo que también juega al shogi y ajedrez. Es interesante analizar cómo los creadores de AlphaGo han logrado esta generalización del algoritmo. En concreto, la red neuronal en la que está basado, se actualiza constantemente, lo que permite en cierta manera que el modelo se adapte en los primeros movimientos al sistema de juego. ¿Podría ser ésto una forma de ir accediendo a una inteligencia artificial general?



vectorjuice (https://www.freepik.es/vector-gratis/chatbot-ilustracion-concepto-abstracto-salud_11667600.htm#query=weak%20artificial%20intelligence&position=19&
(CC BY-SA (http://creativecommons.org/licenses/?lang=es))



dronepicr (https://www.flickr.com/photos/132646954@N02/36851382835/in/album-72157685789962893/) (CC BY (http://creativecommons.org/licenses/?lang=es))

Coches autónomos

Un vehículo autónomo parece una unidad compleja que es capaz de desarrollar varias tareas muy diferentes y puede tener la apariencia de se una inteligencia artificial fuerte o general, pero no. En un coche autónomo, existen diferentes sistemas que se hacen cargo de diferentes tareas, cada uno de la suya, y que son coordinados o controlados por un módulo de control. Cada parte del sistema es una inteligencia artificial débil, controlados por otra inteligencia artificial débil especializada en llevar a cabo ese control, de esos sistemas concretos.

Algoritmos de recomendación

El acto de recomendar una película, un libro o un sitio que visitar parece tan humano que hace difícil clasificar este tipo de sistemas como inteligencia artificial débil. Tras una recomendación suele haber vivencias y experiencias muy variadas, entremezcladas con diversas acciones y hechos. Y lo cierto es que estos sistemas se entrenan con conjuntos de datos que configuran, con un cierto grado de simplificación, los distintos perfiles de gustos de las personas según ciertos parámetros de experiencias registradas.

Autoevaluación

Hay aplicaciones de inteligencia artificial débil reales, no sólo en el cine o la ciencia ficción.

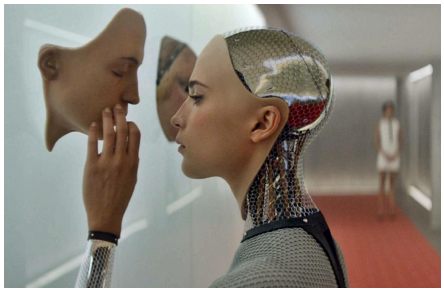
○ Verdadero ○ Falso

Verdadero

Es verdadero. Toda la parte de machine learning es una rama de la IA débil

1.2.- Inteligencia Artificial Fuerte

El concepto de Inteligencia Artificial Fuerte o General proviene más de la imaginación o de la ciencia ficción que de una arquitectura o concepto factible real. Podemos decir que es el punto al cual queremos llegar en este campo, pero no termina de estar claro si es posible con los algoritmos actuales.
El concepto de la Inteligencia Artificial Fuerte o General parte de la posibilidad de que una máquina pueda abstraer hasta el nivel de interaccionar con seres humanos y con el resto de objetos y seres del mundo físico en todo tipo de contexto y con objetivos de todo tipo. En definitiva, se quiere emular la capacidad de interpretación y adaptación que tiene el ser humano. Pero, como ya se ha mencionado en secciones anteriores, no se trata de imitar o comportarse como los humanos, sino de poder acometer las tareas necesarias para múltiples contextos, y, hacerlo igual o mejor que los humanos.



Fotograma película Ex Machina (https://www.filmaffinity.com/ar/film132582.html) (CC0 (http://creativecommons.org/publicdomain/zero/1.0/deed.es))

Aproximación filosófica al concepto de inteligencia artificial fuerte

La posibilidad real de la Inteligencia Artificial Fuerte se ha empezado a estudiar sobre todo a partir de que el filosofo Searle presentara un argumento en contra a ésta en la publicación «The Chinese Room Argument» así como a raíz del argumento del primer teorema de incompletitud de Gödell, que afirma que nunca una máquina podría adquirir algún tipo de inteligencia humana. Los argumentos en contra a la IA fuerte no han dejado de aparecer, como por ejemplo el filósofo heideggeriano Hubert Dreyfus, que dedicó gran parte de su obra argumentando cómo la IA llegaría a fallar porque simplemente el cerebro humano y mente humana no son análogas con el hardware y software de un ordenador. Y sin duda, nuevos estudios demuestran que la mente humana no es como un ordenador.
A partir de este punto, hay dos razones que hacen que tenga una mayor consideración la IA (Inteligencia Artificial) débil:

- Desde la vertiente lógica, se pueden estudiar los algoritmos y las funciones matemáticas con mejor claridad;
- Desde una perspectiva clásica de la filosofía continental, la debilidad intrínseca de la IA se podría argumentar a partir de la siguiente cuestión: nosotros, los humanos, fabricamos las herramientas y sistemas y, por tanto, incluye dentro nuestra propia fragilidad, la posibilidad del fallo.

Como dice Jean Francois Dortier en "La cerveau et la pensée":
«Cuarenta años después de su creación, la importancia de la IA está, cuando menos, mitigada. Los especialistas se refieren cada vez más (después del análisis de Searle) al proyecto de una «IA débil» opuesto a la «IA fuerte» del principio. El proyecto de la «IA fuerte» era encontrar y reconstruir la forma en que el hombre piensa y después superarla. El proyecto de la «IA débil» es más modesto. Se trata de simular comportamientos humanos «considerados inteligentes» por los métodos de la ingeniería, sin preocuparse de saber si el hombre procede de la misma manera. Hoy día se prefiere hablar de lógicas «de ayuda» a la creación o a la toma de decisiones más que de máquinas que remplazarían al hombre».

Aproximación técnica a una arquitectura multitarea

Más allá de que realmente sea posible construir un modelo con las capacidades de conciencia, sensibilidad, sabiduría y autoconocimiento, lo que sí se está explorando, por el momento, es la construcción de modelos, basados en redes neuronales profundas, que sean capaces de ejecutar tareas en diferentes contextos.



Deepmind (https://www.deepmind.com/publications/a-generalist-agent) (Dominio público)

Deepmind, a mediados de 2022, anunció un posible avance hacia una IA general con su nuevo modelo "Gato". Según anunció el laboratorio de inteligencia artificial propiedad de Alphabet, este modelo puede jugar videojuegos de Atari, subtitular imágenes, chatear y apilar bloques con un brazo robótico real. En general, Gato puede realizar hasta 604 tareas diferentes. Lo malo es que Gato no realiza las tareas tan bien como aquellos modelos que solo pueden hacer una cosa.
Gato es un modelo "generalista" en el sentido que puede hacer muchas cosas diferentes al mismo tiempo. Pero eso es muy distinto a una IA "general" capaz de adaptarse de manera eficaz a nuevas tareas distintas a aquellas en las que el modelo fue entrenado.

Conciencia artificial

El paso final del desarrollo de la IA es construir sistemas que puedan formar representaciones sobre sí mismos. En última instancia, los investigadores de la IA tendrán que comprender no solo la conciencia, sino también construir máquinas que la tengan.
Nada garantiza que el desarrollo de una IA fuerte o general implique el surgimiento de la conciencia artificial o que se genere una conciencia emergente, pero esta tiende a ser considerada como la posibilidad más plausible. La razón por la cual se cree que la existencia de uno de estos conceptos probablemente lleve al otro deriva de como de intrínsecamente entrelazadas se encuentran la conciencia y la inteligencia general en los seres vivos.
Peter Voss, profesional del campo de la IA fuerte, justifica la coincidencia fundamental de las cualidades necesarias para el desarrollo tanto de un sistema de IA fuerte como de una conciencia artificial afirmando que las personas poseen "autoconciencia conceptual" (conceptos abstractos del "yo" físico y mental), característica que resultaría imprescindible en dicha IA, ya que esta tendría que "poder conceptualizar qué acciones ha tomado, de qué acciones es capaz, y cuáles son sus efectos más probables.

En plena carrera por el mejor algoritmo conversacional, Blake Lemoine, un ingeniero de Google se refirió al sistema de IA en el que trabajaba, LaMDA, como una máquina pensante y sintiente. El LaMDA (Language Model for Dialogue Applications, modelo de lenguaje para aplicaciones de diálogo en español) fue diseñado por Google en 2017 y tiene como base un transformer, es decir, un entramado de redes neuronales artificiales profundas. Para los expertos en Inteligencia Artificial, transformers como LaMDA han supuesto un hito porque "permiten un procesamiento (de información, de textos) muy eficiente y han producido una auténtica revolución en el campo del Procesamiento del Lenguaje Natural".

Pero, aunque la conversación sea fluida, de calidad y específica, no es más que una enorme fórmula que ajusta los parámetros para predecir mejor la siguiente palabra. No tiene ni idea de lo que está hablando. En definitiva: no tiene autoconciencia ni sensibilidad como tales.

Para saber más

Si quieres conocer la arquitectura y funcionamiento del modelo generalista Gato de Deepmind, te recomendamos que leas el artículo en el que se anunciaba "A Generalist Agent (https://www.deepmind.com/publications/a-generalist-agent)" en la web de Deepmind.

Autoevaluación

¿Qué modelo es capaz, actualmente, de llevar a cabo varias tareas en contextos diferentes?

- ○ Gato, de DeepMind
- ○ Dalle2, de openAI
- ○ Watson, de IBM

Correcto: Watson se utiliza solo para análisis y tratamiento de lenguaje natural y Dalle2 solo se puede utilizar para generar imágenes.

Incorrecto: Dalle2 solo se puede utilizar para generar imágenes.

Incorrecto: Watson se utiliza solo para análisis y tratamiento de lenguaje natural.

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

2.- Caracterización de sistemas de aprendizaje automático

Caso práctico



@casfatesvano (https://twitter.com/casfatesvano) (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

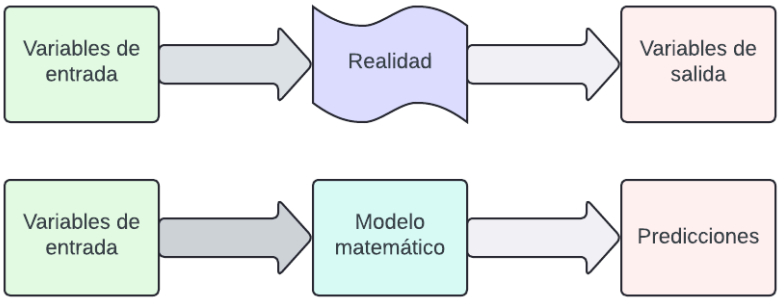
Max está fascinada por las posibilidades que nos presenta la inteligencia artificial. Quiere entrar en materia, pero sobre todo aplicar la IA a problemas reales. Ha descubierto que los algoritmos se entrenan con datos para realizar predicciones, pero todavía no sabe qué clasificación de problemas existe y que tipos de algoritmos concretos puede usar. En esta sección descubrirá qué entendemos por modelo, qué tipos de variables tienen los problemas, así como los tipos de problemas de aprendizaje automático.

Modelo

La realidad es muy compleja y para realizar predicciones basándonos en los datos necesitamos un modelo matemático.

Podemos considerar la realidad como una caja negra, es decir, tiene unas variables de entrada X y unas variables de salida y en medio no sabemos que ha pasado. Esto es así puesto que la realidad es compleja.

Nuestro objetivo en el aprendizaje automático es encontrar un modelo matemático que explique esa realidad. Es decir, que dados unos datos de entrada, prediga de una manera más o menos correcta las variables de salida. Este modelo dependerá de la información que tengamos y se entrenará para que se adecúe a los datos y problema concreto.



Creación propia. *modelo* (CC0 (http://creativecommons.org/publicdomain/zero/1.0/deed.es))

Para construir el modelo que explique la realidad necesitaremos parámetros e hiperparámetros. Los parámetros son los que van ajustando el modelo a la realidad, es decir, cuando entrenemos el modelo son los que se ajustarán al problema y a los datos concretos. Los hiperparámetros construyen una familia de modelos diferentes.

Consideremos que queremos ajustar un polinomio a unos datos. Los valores que acompañan a las x del polinomio son los parámetros y el grado del polinomio es el hiperparámetro. Es decir si el hiperparámetro es 1 tendremos una recta y los parámetros serían que recta de entre todas las posibles explican mejor los datos. Si el hiperparámetro es 2 tendremos una parábola y los parámetros se entrenarían para poder elegir de entre todas las parábolas posibles cuál de ella explica mejor los datos.

Tipología de las **variables**

- Variables cuantitativas
  - Son valores numéricos
  - Pueden ser continuas o discretas
    - Continuas. Valores reales
    - Discretas. Valores enteros finitos
- Variables ordinales
  - Expresan orden
- Variables cualitativas
  - Definen una cualidad como por ejemplo bueno, malo, colores, tipos, etc.

Tipos de **problemas**

En el aprendizaje automático nos encontramos con los siguientes tipos de problemas:

- Problemas de clasificación.
  - La variable a predecir es cualitativa. Por ejemplo, clasificar pisos entre caros y baratos.
- Problemas de regresión.
  - La variable a predecir es continua. Por ejemplo, predecir el precio de un piso.
- Análisis clúster.
  - Tiene como objetivo averiguar la etiqueta de salida que no tenemos (es para aprendizaje no supervisado).
  - Por ejemplo, segmentar clientes.

Autoevaluación

Supongamos que tenemos los datos médicos de pacientes y a cada uno de ellos la etiqueta de si sufre problema cardíaco o no. ¿Qué tipo de problema de aprendizaje supervisado sería?

- ○ Clasificación
- ○ Regresión
- ○ Análisis clúster

Correcto. Es clasificación binaria (elige entre sí y no).

Incorrecto. La variable objetivo es finita y binaria (sí o no)

Incorrecto. Los datos están etiquetados por lo que no puede ser análisis clúster que es un algoritmo de aprendizaje no supervisado

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

2.1.- Aprendizaje supervisado

El aprendizaje supervisado se dedica a resolver problemas de predicción. Tiene como principal característica que los datos van acompañados de sus etiquetas (salidas esperadas).
Por ejemplo, podemos intentar predecir con este tipo de aprendizaje el precio de un coche dadas sus características (siempre que en los datos tengamos además de sus características la etiqueta de salida). En base a eso podemos construir un modelo que infiera que etiqueta le correspondería (precio) a un coche determinado, el cual no esté previamente en nuestros datos.
Con el aprendizaje supervisado podemos resolver problemas de clasificación y de regresión.
Problemas de **clasificación**:

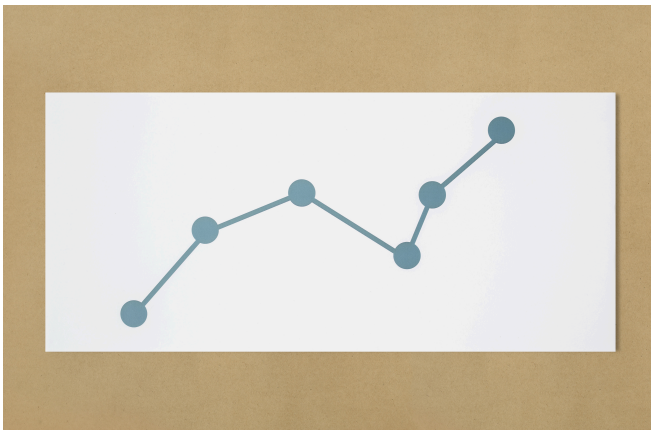
- La variable a predecir es un valor discreto. Se disponen de datos etiquetados con ese valor discreto.
- El objetivo es clasificar las nuevas instancias en un tipo concreto.
- Por ejemplo, clasificar objetos en diferentes tipos.
- Para evaluar estos modelos lo haremos por el porcentaje de elementos clasificados correctamente. Es decir, como tenemos los datos etiquetados, intentamos predecir a que clase corresponden y vemos si hemos acertado.



freepik (https://www.freepik.es/vector-gratis/conjunto-contenedores-reciclaje-basura-ilustraciones-vectoriales-clasificacion-contenedores-basura-coleccion-dibujos-animados-basura-papel-organico-metal-vidrio-plastico-cubos-basura-aislados-blanco-concepto-medio-ambiente_26876949.htm#fromView=search&page=1&position=26&uuid=03e7cc96-cb80-498a-8a93-63247c699e78). *clasificación* (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Problemas de **regresión**:

- La variable a predecir es continua. La etiqueta o bien es un valor real como el precio de un coche o de una acción o bien es un valor no finito como por ejemplo unidades de producto vendidas.
- Para evaluar estos modelos se usan métricas de error como el error absoluto medio (MAE).



freepik (https://www.freepik.es/foto-gratis/icono-analisis-datos-grafico-linea_2826750.htm#fromView=search&page=1&position=2&uuid=e3c48e34-722f-4cd2-96f6-646c374604a2). *regresión* (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Algoritmos más usados para resolver problemas de aprendizaje supervisado:

- Naive Bayes
- Vecinos más cercanos kNN
- Árboles de decisión
- Regresión logística y regresión lineal
- etc.

Para saber más

Aprendizaje supervisado en Scikit-learn:
Aprendizaje supervisado (https://scikit-learn.org/stable/supervised_learning.html)

Autoevaluación

Si disponemos de datos junto con su etiqueta objetivo, estamos ante:

- ◯ Aprendizaje supervisado
- ◯ Aprendizaje no supervisado

Opción correcta

Incorrecto

Solución

1. Opción correcta
2. Incorrecto

2.2.- Aprendizaje no supervisado

Hay que destacar que estos algoritmos son especialmente útiles en estudios de mercado.
En el aprendizaje no supervisado, a diferencia del supervisado, no disponemos de datos etiquetados. Los datos son solo una serie de características de cada observación.
El objetivo principal del aprendizaje no supervisado es realizar agrupamientos en los datos, es decir, dividir los datos en distintos grupos.
Por tanto, el objetivo sería encontrar los grupos que tengan dentro los datos más homogéneos posibles y más heterogéneos con los otros grupos.
Un ejemplo de problema a resolver con este análisis clúster es la segmentación de clientes.
Supongamos que tenemos una serie de características de clientes, pero no los tenemos etiquetados o clasificados. Con el análisis clúster conseguiríamos tener divididos esos clientes de manera automática. La semántica de esos grupos sí sería tarea nuestra. Es decir, el algoritmo nos da los datos agrupados, pero nosotros viendo los datos decidimos qué tipo son por ejemplo clientes buenos o malos, clientes de un sector u otro, etc.
El algoritmo utilizado se denomina k-means y sus pasos son los siguientes:

- Inicialización: Elegir aleatoriamente los centroides (https://es.wikipedia.org/wiki/Centroide) para los k grupos
- Asignación: Calcular la distancia de cada uno a los centroides y asignar los objetos a los grupos
- Actualización de los centroides: Calcular los nuevos centroides (media)
- Si no hay cambio en los centroides se para, si no, se va otra vez al paso de asignación



freepik (https://www.freepik.es/foto-gratis/arreglo-peones-coloridos-alto-angulo_25629285.htm#fromView=search&page=2&position=28&uuid=ced93098d-48b0-a1eb-29d4469b48e1), *grupos* (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Para saber más

Visualización de agrupaciones de K-Means (https://www.naftaliharris.com/blog/visualizing-k-means-clustering/)

Autoevaluación

Si disponemos de una serie de datos que no están etiquetados, estamos ante:

- ○ Aprendizaje supervisado
- ○ Aprendizaje no supervisado

Incorrecto

Opción correcta

Solución

1. Incorrecto
2. Opción correcta

3.- Herramientas para el aprendizaje automático

Caso práctico



@casfatesvano (https://twitter.com/casfatesvano) (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Max ya conoce la visión general de problemas de aprendizaje automático a resolver. Sin embargo, quiere aplicarlo de manera práctica y no quedarse solo en pensamientos teóricos. Por tanto, necesita conocer las herramientas reales que se usan para resolver este tipo de problemas.

Python es lenguaje de programación más usado para los sistemas de aprendizaje automático, también conocido como Machine Learning.
Por ello en los siguientes subapartados se dará una visión general de como funciona esta herramienta.
Además para usar todos la misma versión de python y no tener que instalar nada, Google nos facilita la herramienta colab, en la cual podremos ejecutar código python de manera cómoda y sencilla.

Para saber más

Google Colaboratory:

Te damos la bienvenida a Colab (https://colab.research.google.com/?hl=es)

3.1.- Introducción a Python



freepik (https://www.freepik.es/vector-gratis/ilustracion-concepto-mecanografia-codigo_10259340.htm#fromView=search&page=1&position=4&uuid=bc12597e-bbaf-4b57-9897-03fe7385d282). *code* (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Resumen del lenguaje de programación Python.

- Python es un lenguaje interpretado.
- Solo se permite una instrucción por línea.
- No hay que marcar el final de la instrucción.
- El código debe estar bien tabulado.
- No hay que declarar variables.
- Para los identificadores con varias palabras se usa el guion de subrayado.
- None representa el objeto nulo.
- No hay diferencia entre comillas simples y dobles.
- Podemos definir métodos con parámetros opcionales.

Entrada de teclado

La entrada de teclado se hace con la función input(mensaje)

In [2]:
```
nombre = input("Introduce tu nombre ")
apellido = input("Introduce tu apellido ")
print("Hola", nombre, apellido)
```

```
Introduce tu nombre Luke
Introduce tu apellido Skywalker
Hola Luke Skywalker
```

Operadores

Los operadores son los habituales en los lenguajes de programación:

- Suma resta: +, -
- Multiplicación, división, módulo: *, /, %
- División truncada: //
- Exponente: **
- Suma/resta y asignación: +=, -=
- Multiplicación/división y asignación: *=, /=
- Exponente/división truncada y asignación: **=, //=
- Operadores lógicos: or, and, not
- Operadores de comparación: <, <=, >, >=, !=, ==

In [3]:
```
# ejemplo de uso de operadores

a = 3 + 3 - 2 + 5 * 5
a += 1
a -= 10
print(a)
```

```
20
```

In [4]:
```
# division resto exponente
division_real = 3 / 2
division_entera = 3 // 2
resto_division = 3 % 2
potencia = 3 ** 2

print("Resultados: ", division_real, division_entera, resto_division, potencia)
```

```
Resultados:  1.5 1 1 9
```

In [5]:
```
# operadores de comparacion
print(3 > 5)

# operadores lógicos
print(9 > 5 and 3 < 5)
```

```
False
True
```

Sentencias de control

- if
- while
- for

In [6]:
```
# ejemplo if

x = 4
if x < 0:
  print("Negativo")
elif x == 0:
  print("Cero")
else:
  print("Positivo")
```

```
Positivo
```

In [7]:
```
# ejemplo while

i = 1
while i < 6:
  print(i)
  i += 1
```

```
1
2
3
4
5
```

In [8]:
```
# ejemplo for

mascotas = ['gato', 'perro', 'pez']
for p in mascotas:
  print(p, len(p))
```

```
gato 4
perro 5
pez 3
```

In [10]:
```
# ejemplo de uso de funcion range en for
# range([start,] stop [,step])

for i in range(3):
  print(i)
```

```
0
1
2
```

In [11]:
```
for i in range(5, 10):
  print(i)
```

```
5
6
7
8
9
```

In [12]:
```
for i in range(1, 10, 2):
  print(i)
```

```
1
3
5
7
9
```

Cadenas de caracteres

In [13]:
```
cadena1 = "Las cadenas de caracteres"
cadena2 = 'pueden estar entre comillas simples'
cadena3 = "o dobles"

print(cadena1, cadena2, cadena3)
```

```
Las cadenas de caracteres pueden estar entre comillas simples o dobles
```

In [14]:
```
# \n es salto de línea
print('C:\docs\nombre')

# raw
print(r'C:\docs\nombre')
```

```
C:\docs
ombre
C:\docs\nombre
```

In [15]:
```
# tres comillas -> tal y como se ve
print("""
Hola
    Esto es un saludo    A
    Fin del saludo       B
""")
```

```
Hola
    Esto es un saludo    A
    Fin del saludo       B
```

In [16]:
```
# concatenación de cadenas con +
hola_mundo = "hola" + " " + "mundo"
print(hola_mundo)
```

```
hola mundo
```

In [18]:
```
# repetición de cadenas
hola3 = 3 * "hola"
print(hola3)
```

```
holaholahola
```

In [19]:
```
# se pueden indexar con []
letra = hola_mundo[3]
ultima_letra = hola_mundo[-1]
print(letra)
print(ultima_letra)
```

```
a
o
```

In [21]:
```
# slice
s = hola_mundo[2:7]
mundo = hola_mundo[5:]
hola = hola_mundo[:4]

print(s)
print(mundo)
print(hola)

# longitud de una cadena
n = len(hola)
print(n)
```

```
la mu
mundo
hola
4
```

In [22]:
```
# slice
# +---+---+---+---+---+---+
# | P | y | t | h | o | n |
# +---+---+---+---+---+---+
# 0   1   2   3   4   5   6
# -6  -5  -4  -3  -2  -1
```

Autoevaluación

En Python no es importante tener correctamente tabulado el código

◯ Verdadero ◯ Falso

Falso

Las cadenas de caracteres en Python se pueden encerrar entre comillas simples o dobles

◯ Verdadero ◯ Falso

Verdadero

3.2.- Estructuras de datos en Python



freepik (https://www.freepik.es/vector-gratis/ilustracion-concepto-mecanografia-codigo_10259340.htm#fromView=search&page=1&position=4&uuid=bc12597e-bbaf-4b57-9897-03fe7385d282). *code* (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Estructuras de datos en Python:

- Listas.
- Uso de listas como pilas.
- Tuplas.
- Conjuntos.
- Diccionarios.

Listas

Las listas almacenan una serie de datos:

- Están ordenadas.
- Pueden contener objetos.
- Se accede a cada elemento por su posición.
- Un elemento podría ser a su vez otra lista.
- Son de tamaño variable.
- Son mutables (los objetos pueden cambiar).

In [ ]:
```
# las listas se representan entre corchetes y separados por comas
lista = [1, 2, 'cadena', 4]
print(lista)
```

```
[1, 2, 'cadena', 4]
```

In [ ]:
```
lista = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
len(lista)
```

Out[ ]:
```
10
```

In [ ]:
```
# primer elemento
lista[0]
```

Out[ ]:
```
1
```

In [ ]:
```
# último elemento
lista[-1]
```

Out[ ]:
```
10
```

In [ ]:
```
# cuatro primeros elementos
lista[0:4]
```

Out[ ]:
```
[1, 2, 3, 4]
```

In [ ]:
```
# todos los elmentos menos los 4 últimos
lista[:-4]
```

Out[ ]:
```
[1, 2, 3, 4, 5, 6]
```

In [ ]:
```
# lista[inicio:fin:paso]
lista[0:8:2]
```

Out[ ]:
```
[1, 3, 5, 7]
```

Funciones de las listas:

- append(elemento) añade un elemento a la lista.
- pop(indice) elimina un elemento de la lista.
- extend(lista) añade los elementos de una lista a otra.
- remove(elemento) elimina un elemento de la lista.
- index(elemento) devuelve la posición del elemento.
- insert(indice, elemento) inserta un elemento en una posición.
- clear() elimina todos los elementos de la lista.
- count(elemento) devuelve el número de veces que aparece el elemento en la lista.
- sort() ordena los elementos de la lista.
- reverse() invierte los elementos de la lista.
- copy() devuelve una copia de la lista.

In [ ]:
```
lista = [1, 2, 3, 4]
lista.append(5)
print(lista)
```

```
[1, 2, 3, 4, 5]
```

In [ ]:
```
# eliminar y devolver el primer elemento
elemento = lista.pop(0)
print(elemento)
print(lista)
```

```
1
[2, 3, 4, 5]
```

In [ ]:
```
# eliminar y devolver el último elemento
elemento = lista.pop()
print(elemento)
print(lista)
```

```
5
[2, 3, 4]
```

In [ ]:
```
# concatenar listas
lista2 = [5, 6, 7]
lista.extend(lista2)
print(lista)
```

```
[2, 3, 4, 5, 6, 7]
```

In [ ]:
```
# eliminar elemento
lista.remove(6)
print(lista)
```

```
[2, 3, 4, 5, 7]
```

In [ ]:
```
# posicion del elemento
pos = lista.index(5)
print(pos)
```

```
3
```

In [ ]:
```
# insertar elementos
lista.insert(1, 5.3)
print(lista)
```

```
[2, 5.3, 3, 4, 5, 7]
```

In [ ]:
```
# borrar la lista
lista.clear()
```

Usando listas como pilas

Para añadir elemento al tope de la pila usamos append(elemento) Para eliminar elemento del tope de la pila usamos pop().

In [ ]:
```
pila = [3, 4, 5]
pila.append(6)
pila.append(7)
print(pila)

tope = pila.pop()
print("elemento sacado", tope, "pila", pila)

tope = pila.pop()
print("elemento sacado", tope, "pila", pila)
```

```
[3, 4, 5, 6, 7]
elemento sacado 7 pila [3, 4, 5, 6]
elemento sacado 6 pila [3, 4, 5]
```

Más ejemplos de listas

In [ ]:
```
cuadrados = [] # lista vacía

# añadimos los cuadrados del 0 al 9
for x in range(10):
  cuadrados.append(x**2)

print("cuadrados", cuadrados)

# equivalente
cuadrados2 = [x**2 for x in range(10)]
print("cuadrados2", cuadrados2)
```

```
cuadrados [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
cuadrados2 [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

In [ ]:
```
combs = []
for x in [1,2,3]:
  for y in [3,1,4]:
    if x != y:
      combs.append((x,y))
print(combs)


# equivalente
combs2 = [(x, y) for x in [1,2,3] for y in [3,1,4] if x!=y]
print(combs2)
```

```
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

In [ ]:
```
# iterar
questions = ['name', 'quest', 'favorite color']
answers = ['lancelot', 'the holy grail', 'blue']

for q, a in zip(questions, answers):
  print('What is your {0}? It is {1}.'.format(q, a))
```

```
What is your name? It is lancelot.
What is your quest? It is the holy grail.
What is your favorite color? It is blue.
```

Tuplas

Una tupla es una lista inmutable (una vez creada no se puede ni modificar ni añadir o eliminar elementos):

- Son más rápidas que las listas.
- Protegen los datos de cambios accidentales.
- Se pueden usar como claves para diccionarios.
- Usan paréntesis para delimitar sus elementos.

In [ ]:
```
una_tupla = (1, 2, 3)
print(una_tupla)
print(una_tupla[0])
```

```
(1, 2, 3)
1
```

In [ ]:
```
# no se pueden modificar (esto dará error)
una_tupla[0] = 56
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
 in ()
      1 # no se pueden modificar (esto dará error)
----> 2 una_tupla[0] = 56

TypeError: 'tuple' object does not support item assignment
```

In [ ]:
```
tupla = (1,2,3)
print('primer elemento', tupla[0], 'tupla', tupla)

tupla2 = 1,2,3
print('primer elemento', tupla2[0], 'tupla', tupla2)

# tupla con un único elemento
tupla3 = 5,
print('primer elemento', tupla3[0], 'tupla', tupla3)

# empaquetado de tuplas
t = 12345, 54321, 'hola'
print('empaquetado', t)

# desempaquetado
x, y, z = t
print('desempaquetado', x, y, z)
```

```
primer elemento 1 tupla (1, 2, 3)
primer elemento 1 tupla (1, 2, 3)
primer elemento 5 tupla (5,)
empaquetado (12345, 54321, 'hola')
desempaquetado 12345 54321 hola
```

Conjuntos

Un conjunto es una colección no ordenada y sin elementos repetidos. Tienen operaciones matemáticas (union, intersección, diferencia, diferencia simetrica).

In [ ]:
```
# conjunto
basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
print('los duplicados son eliminados', basket)

print('está orange en basket', 'orange' in basket)
print('está fresa en basket', 'fresa' in basket)
```

```
los duplicados son eliminados {'apple', 'pear', 'orange', 'banana'}
está orange en basket True
está fresa en basket False
```

In [ ]:
```
a = set('abracadabra')
b = set('alacazam')
print('a', a)
print('b', b)
```

```
a {'c', 'd', 'r', 'b', 'a'}
b {'m', 'c', 'z', 'l', 'a'}
```

In [ ]:
```
diferencia = a - b # letras en a pero no en b
union = a | b # letras en a, en b o en ambos
interseccion = a & b # letras en a y b
diferencia_simetrica = a ^ b # letras en a o b pero no en ambos

print('diferencia', diferencia)
print('union', union)
print('interseccion', interseccion)
print('diferencia simetrica', diferencia_simetrica)
```

```
diferencia {'d', 'r', 'b'}
union {'m', 'c', 'd', 'r', 'b', 'z', 'l', 'a'}
interseccion {'c', 'a'}
diferencia simetrica {'r', 'm', 'l', 'd', 'z', 'b'}
```

Diccionarios

Los diccionarios son una estructura de datos que contienen parejas clave-valor:

- Cada elemento valor tiene asociada una clave.
- Los elementos no están ordenados.
- Las claves no se pueden repetir.
- Para las claves sólo se pueden usar tipos de datos inmutables.

In [ ]:
```
habitantes = {'Granada':919700, 'Sevilla':1950000, 'Malaga':1661000}

h_gra = habitantes['Granada']
print('h_gra', h_gra)
print('Está Sevilla en el diccionario?', 'Sevilla' in habitantes)

# añadir elementos al diccionario
habitantes['Almeria'] = 1000000

print(habitantes)
```

```
h_gra 919700
Está Sevilla en el diccionario? True
{'Granada': 919700, 'Sevilla': 1950000, 'Malaga': 1661000, 'Almeria': 1000000}
```

In [ ]:
```
# creación de diccionario con dos elementos
tel = {'jack': 4098, 'sape': 4139}

# añadior teléfono
tel['guido'] = 4127
print('telefonos', tel)

# consultar el telefono de jack
tel_jack = tel['jack']
print('telefono de jack', tel_jack)

# borrar sape
del tel['sape']
print('telefonos despues de borrar sape', tel)

# añadir irv
tel['irv'] = 4127
print('telefonos despues de añadir a irv', tel)
```

```
telefonos {'jack': 4098, 'sape': 4139, 'guido': 4127}
telefono de jack 4098
telefonos despues de borrar sape {'jack': 4098, 'guido': 4127}
telefonos despues de añadir a irv {'jack': 4098, 'guido': 4127, 'irv': 4127}
```

In [ ]:
```
# recorrer diccionarios

habitantes = {'Granada':919700, 'Sevilla':1950000, 'Malaga':1661000}

for ciudad in habitantes:
  print(ciudad)

for ciudad in habitantes.keys():
  print(ciudad)

for numero in habitantes.values():
  print(numero)

# obtener al mismo tiempo clave y valor
for k, v in habitantes.items():
  print(k, v)
```

```
Granada
Sevilla
Malaga
Granada
Sevilla
Malaga
919700
1950000
1661000
Granada 919700
Sevilla 1950000
Malaga 1661000
```

Autoevaluación

Los datos de una tupla creada en Python se pueden modificar

○ Verdadero ○ Falso

Falso

Son inmutables

Una lista puede usarse como índice en un diccionario

○ Verdadero ○ Falso

Falso

Una lista no se puede usar como índice, pero una tupla si

3.3.- Funciones y clases en Python



freepik (https://www.freepik.es/vector-gratis/ilustracion-concepto-mecanografia-codigo_10259340.htm#fromView=search&page=1&position=4&uuid=bc12597e-bbaf-4b57-9897-03fe7385d282). *code* (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Funciones y clases en Python:

- Funciones.
- Clases.

Funciones

In [1]:
```
# b y c son argumentos con valor por defecto
def sumar_numeros(a, b=0, c=0):
  return a+b+c

def fib(n):
  a, b = 0, 1
  while a < n:
    print(a, end=' ')
    a, b = b, a+b
  print()
```

In [2]:
```
# llamadas a funciones
fib(1000)
```

```
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

In [3]:
```
# las funciones las podemos renombrar (como los punteros a funciones en c)
f = fib
f(1000)
```

```
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

Clases

La programación orientada a objetos en Python tiene algunas diferencias con Java a tener en cuenta:

- Se permite herencia múltiple.
- No hay modificadores de acceso (public, private, etc.).
- No hay sobrecarga de métodos.
  - Para ello usaremos parámetros opcionales.

In [14]:
```
class Pelicula:
  # atributo de clase
  productora = 'Netflix'

  # constructor de la clase
  def __init__(self, titulo, director, presupuesto):
    # atributos de instancia
    self.titulo = titulo
    self.director = director
    self.presupuesto = presupuesto

  # representacion en cadena
  def __str__(self):
    return self.titulo + ' ' + self.director

  def aumentar_presupuesto(self, cantidad):
    self.presupuesto += cantidad
```

In [15]:
```
class Serie(Pelicula):
  # esta clase hereda de Pelicula
  def __init__(self, titulo, director, presupuesto, temporadas):
    super().__init__(titulo, director, presupuesto)
    self.temporadas = temporadas

  def anadir_temporada(self, cantidad=None):
    if cantidad is None:
      self.temporadas += 1
    else:
      self.temporadas += cantidad

  def aumentar_presupuesto(self, cantidad):
    super().aumentar_presupuesto(cantidad * self.temporadas)
```

In [16]:
```
serie = Serie('Juego de tronos', 'un director bueno', 10000000, 8)
print(serie)
```

```
Juego de tronos un director bueno
```

In [17]:
```
serie.aumentar_presupuesto(100)
print(serie.presupuesto)
```

```
10000800
```

In [18]:
```
serie.director = 'desconocido'
print(serie)
```

```
Juego de tronos desconocido
```

Para saber más

Completo tutorial oficial de Python:

El tutorial de Python (https://docs.python.org/es/3/tutorial/index.html)

4.- Preprocesamiento de los datos

Caso práctico



@casfatesvano (https://twitter.com/casfatesvano) (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Max está impaciente por realizar estudios de Machine Learning. Sabe de su potencia a la hora de predecir resultados, pero antes debe conocer una serie de conceptos importantes. Por ello, antes de lanzarse a realizar estudios o resolver problemas de aprendizaje supervisado, le han dicho que los datos son muy importantes, por lo que deben ser manejables y sin errores. Por eso es importante preprocesar los datos y prepararlos para su posterior estudio.
A Max le han hecho especial hincapié en que debe conocer los siguientes conceptos: Outliers, observaciones influyentes y escalamiento de los datos.

Outliers
Los outliers son valores extremadamente grandes o extremadamente pequeños que tienen su origen, normalmente, en errores de los datos.
Para detectar cuando un dato es un outlier lo haremos con el llamado método intercuartiles.
Método intercuantiles: Todo lo que quede fuera de un rango [inferior, superior] es considerado outlier. Estos valores se calculan con las siguientes fórmulas:

- inferior = q1 - k (q3 - q1)
- superior = q3 + k (q3 - q1)

En donde:

- q1 es el primer cuartil de la muestra
- q3 es el tercer cuartil de la muestra
- k es un parámetro que se puede ajustar a 1.5 o 3
  - k = 1.5 (outliers débiles)
  - k = 3 (seguro que son outliers)



freepik (https://www.freepik.es/foto-gratis/etiqueta-engomada-plana-jefe-laico-taza_9640630.htm#fromView=search&page=1&position=2&uuid=1e2acc77-b737-4c78-9c22-4037ebb5fcde). *outliers* (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Observaciones **influyentes**:
Las observaciones influyentes son un tipo especial de outlier que influye en el resultado o estudio que estemos haciendo. Por lo que es interesante que cuando detectemos observaciones influyentes las eliminemos de nuestros datos.
Para detectar observaciones influyentes lo haremos con el método de Jackknife que funciona de la siguiente manera:

- Se realiza el estudio quitando un dato cada vez
- Con esos datos (Datos - dato[i]) calculamos la medida
- Como la medida cambia determinamos los outliers

Escalamiento de los datos:
Normalmente, el orden de magnitud de las variables influyen en los procedimientos estadísticos, con el escalamiento de los datos conseguimos que todos tengan la misma importancia.
Por otro lado, para trabajar con gran cantidad de datos, una herramienta útil es la librería **pandas** de python. Esta herramienta permite trabajar con dataframes.
En los siguientes apartados se tratará un pequeño tutorial para trabajar con dataframes y se pondrán ejemplos prácticos para detectar outliers con el método intercuartiles, observaciones influyentes con el método de Jackknife y escalar los datos con MinMaxScaler.

Para saber más

Guía de usuario de pandas (https://pandas.pydata.org/docs/user_guide/index.html)
Método de Jackknife (https://es.wikipedia.org/wiki/Jackknife_(estad%C3%ADstica))
Escalamiento con MinMaxScaler (https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html)

Autoevaluación

Supongamos que tenemos datos de altura en metros y peso en kg de una seríe de personas. ¿Sería conveniente escalar esos datos?

○ Verdadero ○ Falso

Verdadero

Puesto que están en magnitudes muy diferentes

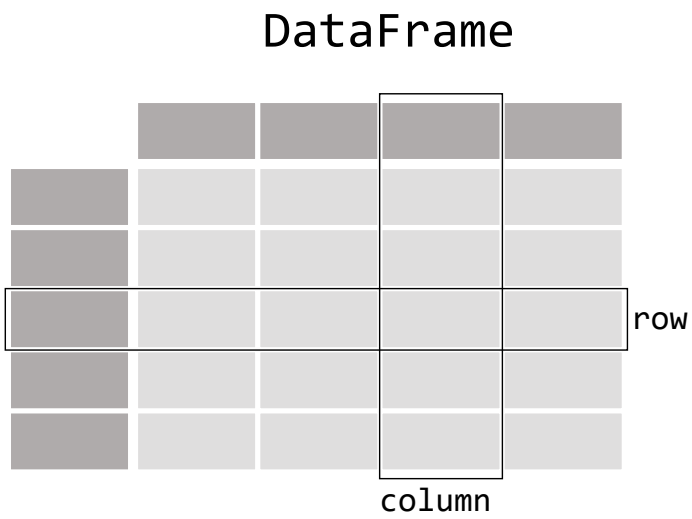
Un outlier que sabemos que no es una observación influyente habría que eliminarlo para continuar con nuestro estudio

○ Verdadero ○ Falso

Falso

Si no es una observación influyente no importa que sea outlier

4.1.- Dataframes con pandas



pandas (https://pandas.pydata.org/docs/getting_started/intro_tutorials/01_table_oriented.html), *dataframe* (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

In [1]:
```
import pandas as pd

# Leer csv
titanic = pd.read_csv("https://raw.githubusercontent.com/pandas-dev/pandas/master/doc/data/titanic.csv")
titanic
```

Out[1]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Ei
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	

891 rows × 12 columns

Funciones útiles de pandas:

- head(n) muestra las n primeras filas.
- tail(n) muestra las n últimas filas.
- dtypes muestra una lista con los tipos de datos.
- info información técnica del dataframe.
- shape número de filas y de columnas.

In [2]:
```
titanic.head(3)
```

Out[2]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emb
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	

In [3]:
```
titanic.tail(4)
```

Out[3]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embark
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	

In [4]:
```
titanic.dtypes
```

Out[4]:
```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

In [5]:
```
titanic.info
```

Out[5]:

pandas.core.frame.DataFrame.info

```
def info(verbose: bool | None=None, buf: WriteBuffer[str] | None=None, max_cols: int |
None=None, memory_usage: bool | str | None=None, show_counts: bool | None=None) -> None
```

Print a concise summary of a DataFrame.

This method prints information about a DataFrame including
the index dtype and columns, non-null values and memory usage.

In [6]:
```
titanic.shape
```

Out[6]:
```
(891, 12)
```

In [7]:
```
# seleccionar columnas

ages = titanic['Age']
ages.head()
```

Out[7]:
```
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
Name: Age, dtype: float64
```

In [8]:
```
age_sex = titanic[['Age', 'Sex']]
age_sex.head()
```

Out[8]:

	Age	Sex
0	22.0	male
1	38.0	female
2	26.0	female
3	35.0	female
4	35.0	male

In [9]:
```
# seleccionar filas

above_35 = titanic[titanic['Age'] > 35]
above_35.head()
```

Out[9]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emba
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	
11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103	
13	14	0	3	Andersson, Mr. Anders Johan	male	39.0	1	5	347082	31.2750	NaN	
15	16	1	2	Hewlett, Mrs. (Mary D Kingcome)	female	55.0	0	0	248706	16.0000	NaN	

Seleccionar filas y columnas Para seleccionar filas y columnas se usa loc o iloc:

- La parte antes de la coma son las filas que se desea.
- La parte después de la coma son las columnas.
- loc -> Nombres de columna.
- iloc -> Posiciones.

In [10]:
```
# nombres de los mayores de 35
adult_names = titanic.loc[titanic["Age"] > 35, "Name"]

adult_names
```

Out[10]:
```
1      Cumings, Mrs. John Bradley (Florence Briggs Th...
6                                 McCarthy, Mr. Timothy J
11                               Bonnell, Miss. Elizabeth
13                             Andersson, Mr. Anders Johan
15                       Hewlett, Mrs. (Mary D Kingcome)
                             ...
865                              Bystrom, Mrs. (Karolina)
871       Beckwith, Mrs. Richard Leonard (Sallie Monypeny)
873                            Vander Cruyssen, Mr. Victor
879          Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
885                    Rice, Mrs. William (Margaret Norton)
Name: Name, Length: 217, dtype: object
```

In [11]:
```
t = titanic.iloc[0:, 3:6]
t
```

Out[11]:

	Name	Sex	Age
0	Braund, Mr. Owen Harris	male	22.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0
2	Heikkinen, Miss. Laina	female	26.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
4	Allen, Mr. William Henry	male	35.0
...	...	...	...
886	Montvila, Rev. Juozas	male	27.0
887	Graham, Miss. Margaret Edith	female	19.0
888	Johnston, Miss. Catherine Helen "Carrie"	female	NaN
889	Behr, Mr. Karl Howell	male	26.0
890	Dooley, Mr. Patrick	male	32.0

891 rows × 3 columns

In [12]:
```
# columnas derivadas

t['nueva_columna'] = t['Age'] + 100
t
```

Out[12]:

	Name	Sex	Age	nueva_columna
0	Braund, Mr. Owen Harris	male	22.0	122.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	138.0
2	Heikkinen, Miss. Laina	female	26.0	126.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	135.0
4	Allen, Mr. William Henry	male	35.0	135.0
...	...	...	...	...
886	Montvila, Rev. Juozas	male	27.0	127.0
887	Graham, Miss. Margaret Edith	female	19.0	119.0
888	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	NaN
889	Behr, Mr. Karl Howell	male	26.0	126.0
890	Dooley, Mr. Patrick	male	32.0	132.0

891 rows × 4 columns

In [13]:
```
# columnas derivadas
t['otra'] = t['nueva_columna'] - t['Age']
t
```

Out[13]:

	Name	Sex	Age	nueva_columna	otra
0	Braund, Mr. Owen Harris	male	22.0	122.0	100.0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	138.0	100.0
2	Heikkinen, Miss. Laina	female	26.0	126.0	100.0
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	135.0	100.0
4	Allen, Mr. William Henry	male	35.0	135.0	100.0
...	...	...	...	...	...
886	Montvila, Rev. Juozas	male	27.0	127.0	100.0
887	Graham, Miss. Margaret Edith	female	19.0	119.0	100.0
888	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	NaN	NaN
889	Behr, Mr. Karl Howell	male	26.0	126.0	100.0
890	Dooley, Mr. Patrick	male	32.0	132.0	100.0

891 rows × 5 columns

In [14]:
```
# Cálculos estadísticos (media)
titanic['Age'].mean()
```

Out[14]:
```
29.69911764705882
```

In [16]:
```
# Cálculos estadísticos (media)
titanic[['Age', 'Fare']].mean()
```

Out[16]:
```
Age     29.699118
Fare    32.204208
dtype: float64
```

In [17]:
```
# Cálculos estadísticos (mediana)
titanic[['Age', 'Fare']].median()
```

Out[17]:
```
Age     28.0000
Fare    14.4542
dtype: float64
```

In [18]:
```
# cálculos estadísticos predefinidos
titanic[['Age', 'Fare']].describe()
```

Out[18]:

	Age	Fare
count	714.000000	891.000000
mean	29.699118	32.204208
std	14.526497	49.693429
min	0.420000	0.000000
25%	20.125000	7.910400
50%	28.000000	14.454200
75%	38.000000	31.000000
max	80.000000	512.329200

In [19]:
```
# agrupar (https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html#pandas.DataFrame.groupby)
titanic[['Sex', 'Age']].groupby('Sex').mean()
```

Out[19]:

	Age
Sex	
female	27.915709
male	30.726645

In [20]:
```
# precio medio de la tarifa para cada una de las combinaciones de clase y sexo
titanic.groupby(['Sex', 'Pclass'])['Fare'].mean()
```

Out[20]:
```
Sex     Pclass
female  1         106.125798
        2          21.970121
        3          16.118810
male    1          67.226127
        2          19.741782
        3          12.661633
Name: Fare, dtype: float64
```

In [21]:
```
# contar el número de registros por categoría
titanic.groupby('Pclass')['Pclass'].count()
```

Out[21]:
```
Pclass
1    216
2    184
3    491
Name: Pclass, dtype: int64
```

Para saber más

Documentación de pandas:

Guía de usuario (https://pandas.pydata.org/docs/user_guide/index.html)

4.2.- Ejemplo práctico preprocesamiento datos



freepik (https://www.freepik.es/vector-gratis/ilustracion-concepto-procesamiento-datos_12219361.htm#fromView=search&page=1&position=1&uuid=ba36a734-d1dc-4555-ae31-fde615b6b6a6), *procesamiento datos* (CC BY-SA (http://creativecommons.org/licenses/?lang=es))

Outliers

Los outliers son valores extremadamente grandes o extremadamente pequeños que tienen su origen, normalmente, en errores de los datos. Para detectar cuando un datos es un outlier lo haremos con el llamado método intercuartiles.

- Método intercuantiles: Todo lo que quede fuera de un rango [inferior, superior] es considerado outliers. Estos valores se calculan con la siguientes fórmulas:
  - inferior = q1 - k (q3 - q1)
  - superior = q3 + k (q3 - q1)

En donde:

- q1 es el primer cuartil de la muestra
- q3 es el tercer cuartil de la muestra
- k es un parámetros que se puede ajustar a 1.5 o 3
  - k = 1.5 (outliers débiles)
  - k = 3 (seguro que son outliers)

In [7]:
```
import numpy as np
import pandas as pd
import scipy.stats as st

# generación de datos para ilustrar el ejemplo
# 100 datos de media 100 y desviación típica 20
media = 100
sigma = 10
datos = np.random.normal(media, sigma, 100)
datos
```

Out[7]:
```
array([ 96.55710402, 103.931371  , 100.51896749, 114.64666238,
       102.83240868,  96.51334636, 104.92572957, 104.77773123,
        85.04701832,  78.52368536, 109.11029431, 103.20385675,
       103.3929962 ,  76.64009983, 102.86564988, 126.49442757,
       100.35211345,  90.16039751, 118.51117503,  90.64761546,
        98.48012859, 109.36760791,  85.11589253,  96.83791893,
       102.70923905, 108.89729644,  96.61459883,  85.76630795,
       106.20960372, 109.78618177, 109.25210228, 121.12393163,
       101.73754273, 103.56979848,  82.14410745, 103.4404532 ,
       110.72183852, 108.48838118,  95.08877366, 105.08651987,
       102.74203534,  88.19732735,  89.57050852,  92.66546137,
       100.40707463,  99.87023928,  89.36073516, 111.59553454,
       103.88556394,  92.82058962, 109.51510531,  83.52087372,
       107.40444612, 109.75055942,  94.82253357, 102.17937195,
        74.42935379, 105.12474065,  79.83540829,  86.74567866,
       104.31607581,  98.58128992,  93.44865808,  84.66406802,
       104.14647994,  94.8189065 ,  89.36870164, 109.41112934,
        96.11288754,  97.00340674,  94.65452554,  92.79956884,
       112.53504594, 108.55003128, 112.9111973 , 106.85676827,
        90.33528777, 126.09060096,  87.34101602, 116.68834953,
        90.73667723,  90.36562065, 108.66361824,  99.0824479 ,
        97.31274066,  85.74899234, 106.57557043,  98.89545263,
       105.36157036,  96.28506362, 115.17594515, 111.25336119,
       101.53442904, 103.36264006, 101.60153334, 107.28043144,
        93.26109491, 102.76884853,  98.43030665,  91.46174697])
```

In [8]:
```
# introducimos dos datos que claramente son outliers
datos[20] = 200
datos[50] = 1
```

In [9]:
```
# método basado en cuartiles para detectar outliers

q1 = np.quantile(datos, 0.25)
q3 = np.quantile(datos, 0.75)
k = 1.5 # outliers débiles

# calculamos los extremos
inferior = q1 - k * (q3 - q1)
superior = q3 + k * (q3 - q1)

# todo lo que se quede fuera de ese rango es un outlier
for i in range(len(datos)):
  if datos[i] < inferior or datos[i] > superior:
    print("el dato", i, "con valor", datos[i], "es un outlier")
```

```
el dato 20 con valor 200.0 es un outlier
el dato 50 con valor 1.0 es un outlier
```

Observaciones influyentes

Las observaciones influyentes son un tipo especial de outlier que influye en el resultado o estudio que estemos haciendo. Por lo que es interesante que cuando detectemos observaciones influyentes las eliminemos de nuestros datos.

Para detectar observaciones influyentes lo haremos con el método de Jackknife https://es.wikipedia.org/wiki/Jackknife_(estad%C3%ADstica (https://es.wikipedia.org/wiki/Jackknife_(estad%C3%ADstica)).

El método de Jackkife funciona de la siguiente manera:

- Se realiza el estudio quitando un dato cada vez.
- Con esos datos (Datos - dato[i]) calculamos la medida.
- Como la medida cambia determinamos los outliers.

In [14]:
```
# ejemplo tenemos precios de coches entre 20000 y 40000 euros
# imaginemos que un coche vale 1 millón
# la idea sería detectar que ese coche es una observación influyente para la media

import numpy as np

# generamos datos aleatorios entre 20000 y 40000
datos = 20000 + np.random.rand(100) * 10000

# a uno de ellos le ponemos el valor de 1 millón
datos[10] = 1000000

# calculamos la media
media = np.mean(datos)
print("La media es:", media)
```

```
La media es: 34409.60981769225
```

In [15]:
```
# realizamos el estudio estadístico quitando un elemento cada vez
medias = np.zeros(len(datos))

for i in range(len(datos)):
  datos_sin_i = np.delete(datos, i)
  medias[i] = np.mean(datos_sin_i)

medias
```

Out[15]:
```
array([34492.92899702, 34538.5816892 , 34525.1624156 , 34550.19555606,
       34493.66124008, 34489.89748573, 34542.33888669, 34540.58477827,
       34465.53033221, 34525.20080564, 24656.17153302, 34501.53010544,
       34477.18069605, 34503.90720863, 34522.2105797 , 34550.99359093,
       34470.58249346, 34474.61572413, 34486.65010087, 34513.04409453,
       34484.49660879, 34491.97812377, 34473.32761326, 34541.62724468,
       34543.73213918, 34543.90226233, 34525.14532844, 34493.22892375,
       34492.15201495, 34491.87074022, 34517.91847012, 34496.13329756,
       34539.29155331, 34541.24962361, 34455.51530158, 34506.71464323,
       34538.06339537, 34455.4826057 , 34520.32425224, 34552.03189901,
       34488.06576574, 34549.27125948, 34483.47239026, 34524.28897604,
       34544.29107826, 34493.64347527, 34537.69161891, 34466.24520075,
       34487.03443339, 34531.69218865, 34540.78271221, 34495.01160652,
       34459.2617018 , 34523.96723289, 34536.31487665, 34460.94314436,
       34478.75963145, 34523.99213959, 34473.08049333, 34462.12739526,
       34542.97773409, 34515.74584057, 34511.53160087, 34540.17339061,
       34536.77572084, 34514.88812416, 34501.45175681, 34487.71320118,
       34472.44277943, 34528.4717862 , 34522.03073138, 34524.19559162,
       34551.05816137, 34486.05184917, 34509.84928323, 34551.21235819,
       34550.4734971 , 34551.06425583, 34522.83229154, 34458.76469036,
       34499.94095061, 34470.51759495, 34485.26205377, 34516.45660176,
       34486.63222324, 34527.82566689, 34466.34030346, 34474.48606661,
       34458.83903967, 34544.76807995, 34527.82022397, 34510.28878247,
       34475.8186444 , 34490.94225618, 34518.15769677, 34493.50415247,
       34537.93814662, 34487.83261738, 34547.01236749, 34475.80198083])
```

In [17]:
```
# para determinar el outlier de esas medias usamos el método de los cuartiles

q1 = np.quantile(medias, 0.25)
q3 = np.quantile(medias, 0.75)
k = 1.5 # outliers débiles

# calculamos los extremos
inferior = q1 - k * (q3 - q1)
superior = q3 + k * (q3 - q1)

# todo lo que se quede fuera de ese rango es un outlier
for i in range(len(medias)):
  if medias[i] < inferior or medias[i] > superior:
    print("la media", i, "con valor", medias[i], "es un outlier que además es influyente")
```

```
la media 10 con valor 24656.17153302248 es un outlier que además es influyente
```

Escalamiento de los datos

Normalmente el orden de magnitud de las variables influyen en los procedimientos estadísticos, con el escalamiento de los datos conseguimos que todos tengan la misma importancia.

- MinMaxScaler. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html (https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html)

In [20]:
```
from sklearn.preprocessing import MinMaxScaler
datos = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
scaler = MinMaxScaler()
scaler.fit(datos)
datos_escalados = scaler.transform(datos)
print("datos", datos)
print("datos escalados", datos_escalados) # los datos escalados están entre cero y uno
```

```
datos [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
datos escalados [[0.   0.  ]
 [0.25 0.25]
 [0.5  0.5 ]
 [1.   1.  ]]
```

Para saber más

- Método de Jackknife. Jackknife (estadística) (https://es.wikipedia.org/wiki/Jackknife_(estad%C3%ADstica))
- Escalamiento con MinMaxScaler. MinMaxScaler (https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html)

Condiciones y términos de uso de los materiales

Materiales desarrollados inicialmente por el Ministerio de Educación, Cultura y Deporte y actualizados por el profesorado de la Junta de Andalucía bajo licencia Creative Commons BY-NC-SA.

Antes de cualquier uso leer detenidamente el siguente Aviso legal (https://www.juntadeandalucia.es/educacion/gestionafp/documentacion/aviso_legal/aviso_legal_mjda/index.html)

Historial de actualizaciones

Versión: 02.00.00	Fecha de actualización: 11/06/24	Autoría: Gerardo González Pérez
Ubicación: Unidad 1 **Mejora (tipo 1):** Cambio completo de la unidad **Ubicación:** Unidad 1 **Mejora (tipo 3):** Cambio de la unidad 1: Justificación y antecedentes del Aprendizaje Automático de 18 horas y con contenidos 1. Inteligencia Artificial Fuerte y Débil; 2. Antecedentes del Machine Learning a: Unidad 1: Introducción a la Inteligencia Artificial y a los Sistemas de Aprendizaje Automático con contenidos 1. Inteligencia Artificial Fuerte y Débil 2. Caracterización de sistemas de aprendizaje automático 3. Herramientas para el aprendizaje automático 4. Preprocesamiento de los datos Los contenidos actuales son muy superficiales y poco relevantes en la materia. Estos contenidos no preparan un buen punto de partida para afrontar el resto de unidades. La modificación se basa en definir correctamente los términos importantes, mostrar cuales son los problemas que resuelven los sistemas de aprendizaje automático, introducir las herramientas que se usan y finalmente presentar una de las etapas más importantes del aprendizaje automático que es el preprocesamiento de los datos. **Ubicación:** Unidad 1 **Mejora (Mapa conceptual):** Ajustar a los contenidos propuestos **Ubicación:** Unidad 1 **Mejora (Orientaciones del alumnado):** Ajustar a los contenidos propuestos **Ubicación:** Unidad 1 **Mejora (Orientaciones del alumnado):** Ajustar a los contenidos propuestos		

Versión: 01.00.00	Fecha de actualización: 07/07/23
Versión inicial de los materiales.	